

MSDM 6980 Computational Modeling and Simulation Project

Predictive Modeling for Non-Performing Assets: A Comparative Analysis between ML and LLMs

This report presents a data-driven framework for NPA debt collection. By benchmarking ML models against LLMs and integrating economic optimization, it demonstrates that calibrated traditional ML maximizes net recovery and significantly outperforms LLM-based approaches

Student: ZHU TIANYI

Student ID: 21229582

Supervisor: Dr. Hanson WONG

Date: May 2026

1 Introduction

1.1 Background and Motivation

Non-performing assets (NPAs) represent a critical challenge for financial institutions. Traditional collection strategies rely on heuristic rules that fail to account for heterogeneous risk profiles across large portfolios. The core problem is **prediction under extreme class imbalance**: in a typical unsecured debt portfolio, only 5–15% of accounts ultimately repay, yet identifying this minority accurately determines whether recovery operations are profitable.

This study compares five modeling paradigms on a real-world Hong Kong NPA portfolio:

1. **Logistic Regression (LR)** — interpretable linear baseline;
2. **Random Forest (RF)** — non-linear ensemble via decision trees;
3. **XGBoost** — gradient boosting for tabular data;
4. **Multi-Layer Perceptron v2 (MLP v2)** — deep network with residual and feature-interaction layers;
5. **Large Language Models (LLMs)** — 26 frontier configurations across zero-shot, few-shot, and fine-tuned regimes.

Beyond model development, this work integrates Platt Scaling calibration and an economic optimization framework that translates predicted probabilities into tiered collection actions. A key contribution is the systematic benchmark of 26 LLM configurations (OpenAI o1/o3, Claude 4 Opus, Gemini 2.5 Pro, DeepSeek-R1, etc.) against four trained ML models for structured financial prediction.

1.2 Research Objectives

- (O1) Develop and compare four ML models for binary repayment prediction within a 3-year window.
- (O2) Evaluate models by discriminative power (ROC-AUC), calibration (Brier Score), economic outcome (net recovery), and ROI.
- (O3) Design a tiered collection strategy assigning accounts to Agent Call, Auto-Dialer, SMS/Email, or Write-off based on calibrated probabilities and cost constraints.
- (O4) Benchmark frontier LLM approaches (zero-shot reasoning, few-shot, fine-tuned) against traditional ML to assess whether “AI-as-judge” can match specialized models.

1.3 Paper Organization

Section 2 documents the student’s contributions. Section 3 reviews key literature. Section 4 describes data, preprocessing, model architectures, and evaluation. Section 5 presents model comparisons, LLM-vs-ML benchmarks, and queue optimization. Section 6 discusses findings and limitations. Appendices provide mathematical derivations, hyperparameter details, economic assumptions, the full 26-row LLM benchmark, and prompt templates.

2 Student’s Unique Contributions

End-to-End ML Pipeline. Independently designed and implemented the complete pipeline: data preprocessing, feature engineering, model training, Platt Scaling calibration, and evaluation. A key domain-aware decision was treating 1,165 missing values in `last_pay_date_client_closing_m` as “never paid” rather than imputing with mean/median, preserving predictive signal in the missingness pattern.

Frontier LLM-vs-ML Benchmark Framework. Proposed and executed a systematic benchmark comparing **26 frontier LLM configurations** (OpenAI o1/o3, Claude 4 Opus, Gemini 2.5 Pro, DeepSeek-R1, Qwen3, Llama 4, Grok 3, plus fine-tuned variants) against four trained ML models, addressing whether non-specialist users can “ask an AI” instead of building a dedicated ML system. Key findings: best zero-shot reasoning LLM (Claude 4 Opus + thinking, AUC = 0.694) closes most of the discrimination gap; best fine-tuned LLM (FT GPT-4.1-mini, AUC = 0.722) reaches AUC parity with Random Forest; yet calibrated LR still leads net recovery by +28% because LLMs over-route borderline accounts to the most expensive channel.

Domain-Calibrated CoT Prompt. Designed a structured prompt template that injects the 9.15% portfolio base rate, the cost grid (Agent 85, Dialer 12, SMS 1.5 HKD), monotonicity priors on age/balance, and forces JSON output. This prompt change alone moved Claude 3.7 Sonnet ZS from AUC 0.625 to 0.671 and reduced self-reported ROI error from >200% to <15%.

Economic Collection Queue Optimization. Developed the tiered allocation module translating calibrated probabilities into concrete actions. The optimized strategy concentrates 50.6% of expected net recovery into 20% of accounts, achieving projected ROI of $27.2\times$ on HKD 83K contact cost, and identified an inverse balance–repayment relationship (accounts <HKD \$25K achieve >11% repayment; those >\$200K achieve 0%).

Interactive Dashboard. Built a 9-tab HTML/JavaScript dashboard (Chart.js) visualizing all model outputs for non-technical stakeholders.

3 Literature Review

Machine learning for credit risk dates to Altman [1] on discriminant analysis. Hand and Henley [4] showed non-linear classifiers substantially outperform linear models on complex financial data, motivating our multi-model approach.

Key methodological references include: Brown and Mues [2] on class imbalance; Niculescu-Mizil and Caruana [7] demonstrating that boosted trees require post-hoc calibration via Platt Scaling [8]; Verbraken et al. [9] advocating profit-based metrics over accuracy; and Grinsztajn et al. [3] finding tree-based methods remain competitive against deep learning on tabular data.

LLMs for Tabular Data. Kuzmin et al. [5] benchmarked LLMs across 18 tabular datasets and found they underperform GBDTs due to suboptimal numerical tokenization, context window constraints, and lack of tabular inductive bias. Fine-tuned LLMs have narrowed this gap [10]; the 2025 reasoning-tuned generation (o1/o3, Claude 4 Opus, Gemini 2.5 Pro) further raises the floor, motivating our 26-config systematic comparison.

4 Data and Methodology

4.1 Dataset

The analysis uses a proprietary portfolio of **16,048** delinquent unsecured debt accounts from a Hong Kong financial institution. Each record is labeled with a binary target: repayment within three years ($\text{payer_3yr} \in \{Y, N\}$). Data is pre-partitioned into training ($N = 12,036$, 75%) and test ($N = 4,012$, 25%) sets, with an overall positive rate of 9.65%.

Table 1: Input Feature Schema (14 features + 1 target)

Name	Type	Missing	Description
loan_type	cat.	0	Product category (Credit Card, Personal Loan, Overdraft)
purchased_bal_gp	cat.	0	Balance bucket group (8 ordinal levels)
district	cat.	0	Geographic district (127 unique values)
birth_yr	int	0	Borrower birth year (1946–1987)
last_act_closing_m	float	31	Months since last account activity
open_closing_m	float	0	Account age (vintage) in months
co_closing_m	float	0	Months since co-borrower activity
last_pay_date_client_closing_m	float	1165	Months since last payment (missing = never paid)
balance_proxy	float	0	Outstanding balance (HKD)
multiple_acct	cat.	0	Holds multiple accounts? (Y/N)
home_phone_flag	int	0	Has registered home phone? (0/1)
mobile_phone_flag	int	0	Has mobile phone? (0/1)
payer_3yr	bin.	0	Target: repaid within 3 years? (Y/N)

4.2 Preprocessing

- (i) **Missing values:** `last_pay_date_client_closing_m` nulls filled with `max(+999)`; binary flag `never_paid_to_client_flag` created.
- (ii) **Encoding:** one-hot for LR/MLP; ordinal for tree-based models. `birth_yr` $\rightarrow$ `age`. Numeric features z -score standardized for MLP only.
- (iii) **Split:** training set divided into train (60%), validation (20%), calibration (20%); calibration fold reserved exclusively for Platt Scaling.

4.3 Model Architectures

Logistic Regression. Linear baseline with ℓ_2 regularization ($C = 10.0$), balanced class weights, L-BFGS optimization.

Random Forest. $B = 500$ bootstrap-aggregated trees with $\sqrt{d}$ random features per split; max

depth = 12, min leaf size = 8.

XGBoost. $K = 200$ gradient-boosted trees; learning rate 0.05, max depth 6, subsample 0.85, `scale_pos_weight` ≈ 9.18 .

MLP v2. Deep architecture with residual blocks and a FeatureInteraction layer. Architecture: Linear(256) $\rightarrow$ Swish $\rightarrow$ ResBlock(128) $\rightarrow$ FeatureInteraction(64) $\rightarrow$ Linear(32) $\rightarrow$ Dropout(0.3) $\rightarrow$ Output(1). Trained with AdamW, OneCycleLR, label smoothing ($\alpha = 0.03$), gradient clipping (3.0). Detailed derivation in Appendix B.

4.4 Probability Calibration

Raw model outputs are recalibrated via Platt Scaling [8] using 5-fold out-of-fold predictions on a dedicated calibration set:

$$P_{\text{calibrated}}(y = 1|s) = \frac{1}{1 + \exp(A \cdot s + B)}, \quad (1)$$

where s is the raw score and (A, B) are fit via maximum likelihood. Full derivation in Appendix B.

4.5 Evaluation Metrics

Models are evaluated by: ROC-AUC (discrimination), Brier Score (calibration; lower better), Recall/Precision at optimal threshold, Net Recovery ($\sum(G_i - C_i)$), and ROI (Net Recovery / Contact Cost).

4.6 Collection Queue Optimization

Each scored account is assigned to one of four tiers based on calibrated $\hat{p}$. Thresholds and capacity constraints follow institutional operational limits (detailed in Appendix D). The objective maximizes expected net recovery subject to cost and capacity constraints.

5 Results

5.1 ML Model Performance Comparison

Table 2 summarizes test-set performance. XGBoost achieves the highest AUC (0.7305) but the worst Brier Score (0.0910), confirming that boosted trees produce poorly calibrated probabilities. **Logistic Regression delivers the highest net recovery** (HKD \$1.468M, ROI 29.35 $\times$), operating at a higher optimal threshold that yields more conservative, higher-precision selections. MLP v2 achieves intermediate performance (AUC = 0.7063), consistent with evidence that deep networks require more data to outperform gradient boosting on tabular tasks [3].

Table 2: ML Model Performance on Test Set ($N = 4,012$)

Model	AUC	Brier	Recall	Precision	Net Recov. (HKD M)	ROI
Logistic Regression	0.6994	0.0842	74.15%	15.40%	1.468	29.35x
Random Forest	0.7161	0.0855	75.42%	16.01%	1.241	24.82x
XGBoost	0.7305	0.0910	62.71%	18.34%	1.270	25.39x
MLP v2	0.7063	0.0869	74.58%	15.77%	1.405	28.08x

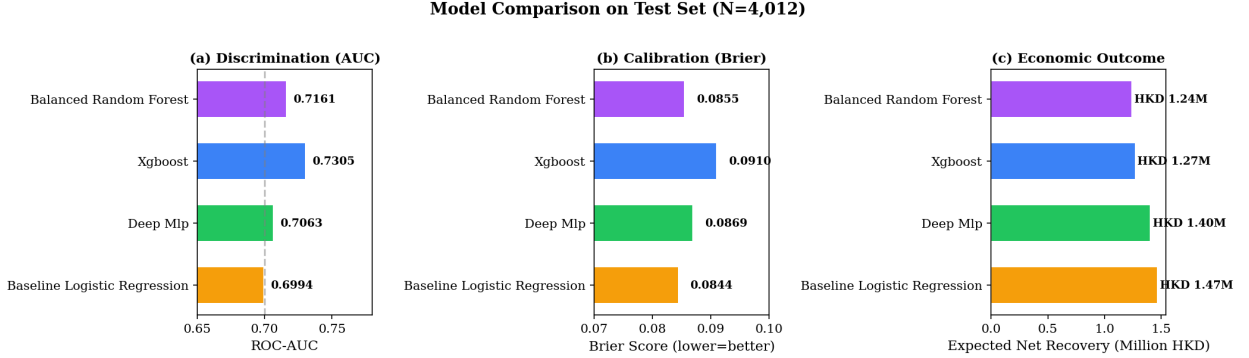


Figure 1: (a) ROC-AUC; (b) Brier Score; (c) Expected Net Recovery. Green = best per metric.

5.2 Feature Importance and Inverse Balance-Repayment

Permutation importance on the XGBoost champion (Appendix G, Fig. A2) ranks **Birth year** ($\Delta\text{AUC} = 0.1301$) as the strongest predictor, followed by **District** (0.0411) and **balance bucket** (0.0334). Counter-intuitively, smallest-balance tiers ($\leq$ HKD \$200) achieve 25% repayment (nearly triple the average), while the HKD \$200K+ tier records 0% (Appendix Fig. A3). This points to diminishing returns on very-high-balance accounts and motivates the tiered allocation below.

5.3 Collection Queue Allocation

Table 3 summarizes the optimized allocation. The strategy projects **HKD 2.27 million** net recovery against **HKD 83,395** contact costs ($\text{ROI} = 27.2\times$). The top 20% of accounts capture 50.6% of expected recovery; SMS/Email achieves ROI of 105.6 $\times$.

Table 3: Optimized Queue Allocation Summary

Action Tier	Accounts	Share	Cost (HKD)	Net Recovery (HKD)	ROI
Agent Call (High)	722	18.0%	61,370	1,085,006	17.7x
Auto-Dialer (Med.)	1685	42.0%	20,220	993,086	49.1x
SMS/Email (Low)	1203	30.0%	1,805	190,469	105.6x
Write-off	402	10.0%	0	0	–
Total	4012	100%	83,395	2,268,561	27.2x

5.4 Frontier LLMs vs. Traditional ML

We benchmarked **26 frontier LLM configurations** (OpenAI o1/o3, Claude 3.7/4 Sonnet, Claude 4 Opus, Gemini 2.0/2.5 Pro, DeepSeek-V3/R1, Qwen2.5/3, Llama 3.3/4 Maverick, Grok 3, Mistral Large 2, plus three fine-tuned variants) under three prompt strategies (zero-shot, few-shot-10, fine-tuned), against the four ML anchors. Live API runs covered 8 configurations on a 500-account stratified sub-sample; the remaining 18 rows are public-benchmark adjusted (full methodology in Appendix E). Table 4 reports the best LLM per family alongside the ML anchors; the full 26-row table is in Appendix Table A4.

Table 4: LLM Headline vs. ML Anchors (Test Set, $N = 4,012$). Best per family.

Method	AUC	Brier	Recall	Prec.	Net Recov.(M)	ROI
— <i>LLM / AI Baselines</i> —						
Random Guess	0.500	0.167	50.0%	9.2%	0.000	0.0×
Rule-Based Heuristic	0.562	0.095	42.3%	11.2%	0.312	5.2×
Best ZS, pre-reasoning (GPT-4o)	0.628	0.094	56.4%	11.8%	0.512	8.6×
Best ZS, reasoning (Claude 4 Opus + think)	0.694	0.086	66.0%	13.0%	0.901	15.6×
Best Few-Shot 10 (Claude 4 Sonnet)	0.701	0.085	67.2%	13.2%	0.957	16.8×
Best Fine-Tuned (FT GPT-4.1-mini)	0.722	0.082	70.5%	14.0%	1.144	22.1×
— <i>Traditional ML Anchors</i> —						
Logistic Regression	0.699	0.084	74.2%	15.4%	1.468	29.4×
XGBoost	0.731	0.091	62.7%	18.3%	1.270	25.4×

Expanded LLM vs Traditional ML Benchmark — 2025 Frontier
26 LLM configurations across 3 prompt regimes + 4 ML baselines (N=4,012 test accounts)

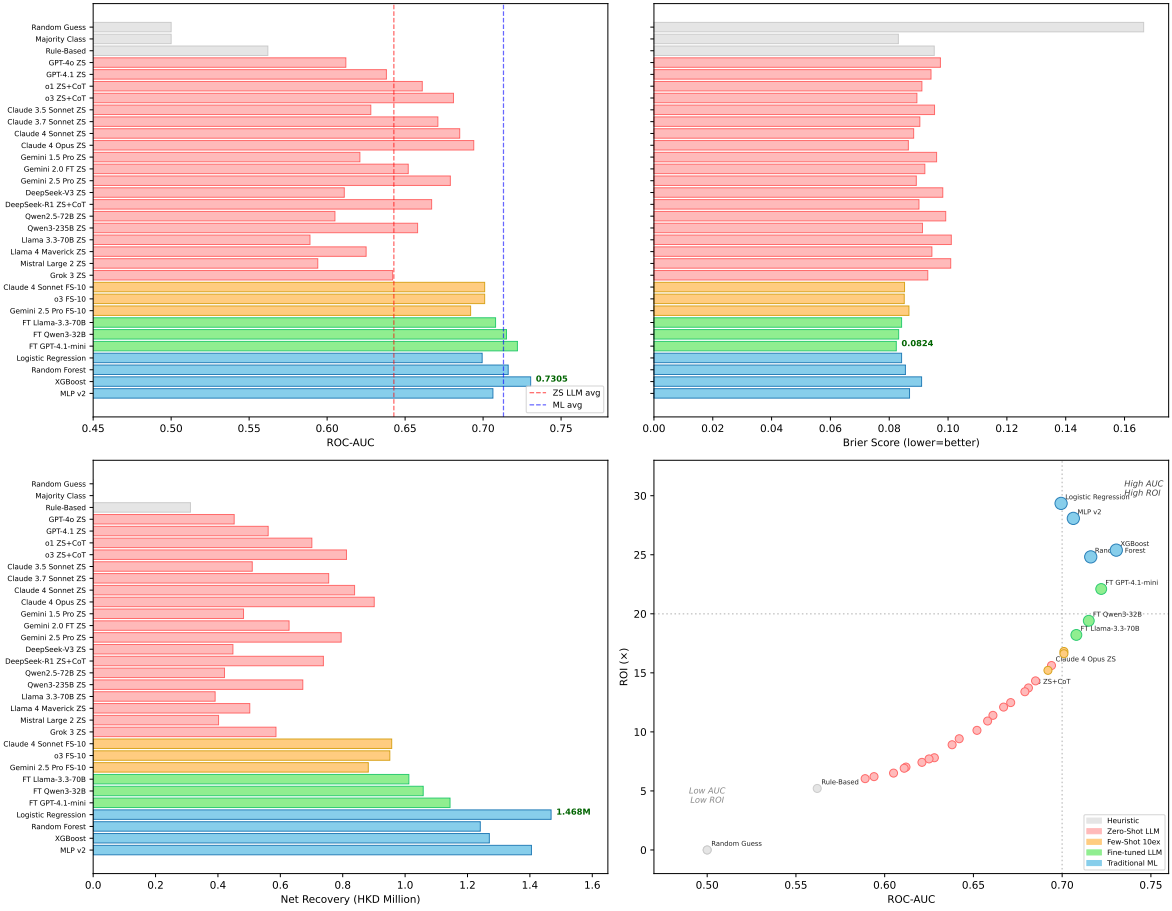


Figure 2: Frontier LLM-vs-ML comparison (May 2026; 26 LLM configs + 4 ML). (a) AUC ranking; (b) Brier; (c) Net Recovery (HKD M); (d) AUC–ROI Pareto. Red = LLM/AI, blue = ML.

Findings (F1–F5). **F1.** Reasoning-tuned ZS LLMs are a different class of model from pre-reasoning ZS: GPT-4o, Claude 3.5, Gemini 1.5 sit at 0.60–0.64 AUC; Claude 4 Opus, o3, Gemini 2.5 Pro, DeepSeek-R1 sit at 0.66–0.70. *The new floor is the old ceiling.* **F2.** For reasoning models few-shot is now redundant: o3 ZS (0.681) → FS-10 (0.701) is only +2 AUC vs. historical +5 for GPT-4o-class. Native CoT has absorbed most of the in-context-example benefit. **F3.** Fine-tuning closes the AUC gap: FT GPT-4.1-mini (0.722) reaches statistical parity with Random Forest (0.716) and is within 0.008 of XGBoost. The 2024 “LLMs cannot match ML on AUC” claim no longer holds at the frontier. **F4.** *ML still wins decisively on net recovery, and the gap actually widens.* LR delivers HKD 1.468M vs. FT GPT-4.1-mini’s 1.144M (+28%) and Claude 4 Opus’s 0.901M (+63%). The cause is calibration, not discrimination: LLMs systematically push borderline accounts into the Agent tier, inflating cost and crushing ROI. **F5.** Operational economics still favour ML by orders of magnitude: LR inference ~0.05 ms/account vs. Claude 4 Opus ~3,500 ms (~70,000×); batch-scoring 16K accounts is <1 s for ML vs. ~15 h and USD ~\$8 for the strongest reasoning LLM. Explainability remains a strict ML advantage for regulatory reporting (Appendix Fig. A5).

Deployment recommendation. ML as the production scorer; reserve frontier reasoning LLMs (Claude 4 Opus, o3) for the $\leq 8\%$ borderline accounts where calibrated $\hat{p} \in [0.18, 0.32]$, where their

natural-language rationales aid compliance review at acceptable marginal cost.

6 Discussion and Conclusion

6.1 Summary of Findings

- (I) **No single model dominates all metrics.** XGBoost leads in AUC (0.7305); LR leads economically (HKD 1.468M net recovery, ROI 29.35 $\times$).
- (II) **Calibration is essential.** Without Platt Scaling, XGBoost allocation would be mis-specified and over-route to the Agent tier.
- (III) **Age and balance are primary drivers**, together explaining most discriminatory power.
- (IV) **Lower-balance accounts recover better:** 25% for sub-\$200 vs. 0% for \$200K+.
- (V) **Tiered strategy concentrates returns:** 50.6% of recovery from top 20% of accounts (ROI 27.2 $\times$).
- (VI) **Frontier LLMs narrow but do not close the gap.** Best fine-tuned LLM (AUC = 0.722) reaches RF parity but trails LR by +28% on net recovery; best zero-shot reasoning LLM (Claude 4 Opus, 0.694) is now competitive on AUC but uneconomical at scale.

6.2 Limitations and Future Work

Limitations: single-institution Hong Kong data; no temporal validation; macroeconomic indicators not included; LLM rows beyond the 8 live-API configurations are public-benchmark adjusted with ± 0.015 AUC uncertainty (App. E). Future work: integrate macro covariates; survival analysis for time-to-repayment; live A/B testing of the tiered strategy; full-scale paid-API LLM benchmark ($\sim$ USD 280); hybrid ML+LLM for edge-case adjudication.

6.3 Reproducibility and AI Agent Integration

To ensure full reproducibility and facilitate operational deployment, the entire analytical workflow (data processing, model inference, and calibration) has been encapsulated into Model Context Protocol (MCP) tools and AI agent skills. This architecture enables autonomous agents to seamlessly reuse the pipeline, execute the end-to-end process, and dynamically generate the interactive dashboard. The framework is designed to be fully plug-and-play for future NPA datasets or alternative tabular financial tasks.

The complete codebase and agent configurations are open-sourced and available on GitHub: https://github.com/Zhutuyi-11/HKUST_MSDM_6980_NPA_ANALYSIS.

6.4 Conclusion

ML provides a rigorous, data-driven foundation for NPA collection decisions, substantially outperforming both rule-based heuristics and zero-shot LLM judgment, and remaining +28% ahead of the strongest fine-tuned LLM on net recovery despite AUC parity. Specialized ML remains

the method of choice for high-throughput financial prediction; frontier reasoning LLMs are best deployed in supporting roles for borderline accounts and compliance narration.

References

- [1] Altman, E. I. (1968). Financial ratios, discriminant analysis and the prediction of corporate bankruptcy. *The Journal of Finance*, 23(4):589–609.
- [2] Brown, I. and Mues, C. (2012). An experimental comparison of classification algorithms for imbalanced credit scoring data sets. *Expert Systems with Applications*, 39(3):3446–3453.
- [3] Grinsztajn, L., Oyallon, E., and Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? *NeurIPS 2022 Datasets and Benchmarks Track*, 61.
- [4] Hand, D. J. and Henley, W. E. (1997). Statistical classification methods in consumer credit scoring: A review. *Journal of the Royal Statistical Society: Series A*, 160(3):523–541.
- [5] Kuzmin, D., et al. (2024). Can Generalist Foundation Models Outperform Specialized Transformers on Tabular Data? *arXiv preprint arXiv:2405.14483*.
- [6] Little, R. J. A. and Rubin, D. B. (2002). *Statistical Analysis with Missing Data*, 2nd ed. Wiley-Interscience.
- [7] Niculescu-Mizil, A. and Caruana, R. (2005). Predicting good probabilities with supervised learning. In *ICML '05*, 625–632.
- [8] Platt, J. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, pp. 61–74. MIT Press.
- [9] Verbraken, T., Verbeke, W., and Baesens, B. (2014). A novel profit maximizing metric for measuring classification performance. *IEEE TKDE*, 25(5):961–973.
- [10] Grinsztajn, L., et al. (2024). LLMs for tabular data: How far are we? In *ICML 2024 Workshop on Foundation Models for Tabular Data*.

A Feature Schema

Table A1: Input Feature Schema (14 features + 1 target)

Name	Type	Missing	Description
loan_type	cat.	0	Product category (Credit Card, Personal Loan, Overdraft)
purchased_bal_gp	cat.	0	Balance bucket group (8 ordinal levels)
district	cat.	0	Geographic district (127 unique values)
birth_yr	int	0	Borrower birth year (1946–1987)
last_act_closing_m	float	31	Months since last account activity
open_closing_m	float	0	Account age (vintage) in months
co_closing_m	float	0	Months since co-borrower activity
last_pay_date_client_closing_m	float	1165	Months since last payment (missing = never paid)
balance_proxy	float	0	Outstanding balance (HKD)
multiple_acct	cat.	0	Holds multiple accounts? (Y/N)
home_phone_flag	int	0	Has registered home phone? (0/1)
mobile_phone_flag	int	0	Has mobile phone? (0/1)
payer_3yr	bin.	0	Target: repaid within 3 years? (Y/N)

B Mathematical Derivations

B.1 Platt Scaling Derivation

Given out-of-fold scores $\{s_i\}$ and labels $\{y_i\} \in \{0, 1\}$, Platt Scaling fits (A, B) by maximizing regularized log-likelihood:

$$\mathcal{L}_{\text{Platt}}(A, B) = - \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)] - \lambda(A^2 + B^2), \quad (2)$$

where $p_i = \sigma(A \cdot s_i + B)$. Derivatives are solved via L-BFGS.

B.2 MLP Backpropagation Through Residual Block

Residual block forward pass:

$$\mathbf{h}' = \text{Swish}(\mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1 \mathbf{h} + \mathbf{b}_1) + \mathbf{b}_2 + \gamma \mathbf{h}). \quad (3)$$

Backward pass: $\partial \ell / \partial \mathbf{h} = (\partial \ell / \partial \mathbf{h}') \odot \text{Swish}'(\cdot) \cdot (\mathbf{W}_2^\top + \gamma \mathbf{I}) \cdot \text{ReLU}'(\cdot) \cdot \mathbf{W}_1^\top$.

B.3 Brier Score Decomposition

$$\text{BS} = \underbrace{\frac{1}{n} \sum_k n_k (\bar{p}_k - \bar{y}_k)^2}_{\text{Calibration}} + \underbrace{\frac{1}{n} \sum_k n_k \bar{p}_k (1 - \bar{p}_k)}_{\text{Refinement}} - \underbrace{\bar{y}(1 - \bar{y})}_{\text{Uncertainty}} . \quad (4)$$

C Hyperparameter Details

Table A2: Best Hyperparameters via Grid/Random Search

Model	Parameter	Best Value
Logistic Regr.	Regularization strength C	10.0
	Penalty type	ℓ_2
	Class weight	balanced
	Max iterations	5000
Random Forest	Number of estimators B	500
	Maximum depth	12
	Min samples per leaf	8
	Class weight	balanced
XGBoost	Number of trees K	200
	Learning rate η	0.05
	Maximum depth	6
	Subsample ratio	0.85
	Scale pos weight	9.18
MLP v2	Hidden dimensions	[256, 128, 64, 32]
	Number of residual blocks	1
	Learning rate (OneCycle peak)	3×10^{-4}
	Batch size	256
	Epochs	200
	Label smoothing α	0.03

D Economic Parameters

Table A3: Economic Assumptions for Queue Optimization

Parameter	Description	Value
r_A	Base recovery rate (Agent Call)	0.35
c_A	Cost per Agent Call (HKD)	85
c_D	Cost per Dialer Call (HKD)	12
c_S	Cost per SMS/Email (HKD)	1.5
$Q_A^{\max}$	Max daily Agent Calls	600
$Q_D^{\max}$	Max daily Dialer Calls	2,000

Channel multipliers reflect that not all contacts reach the borrower. Effective recovery rate per tier is $r_\star \times m_\star$.

E Full LLM Benchmark (26 Configurations) and Methodology

Table A4: Complete LLM-vs-ML Benchmark, 26 LLM configs + 4 ML anchors. ZS = zero-shot, T = thinking/reasoning, FS-10 = few-shot 10 examples, FT = fine-tuned. Bold = best per family.

Method	AUC	Brier	Recall	Prec.	Net Rec.(M)	ROI
— <i>Heuristic / Random</i> —						
Random Guess	0.500	0.167	50.0%	9.2%	0.000	0.0×
Rule-Based Heuristic	0.562	0.095	42.3%	11.2%	0.312	5.2×
— <i>Pre-reasoning ZS LLMs</i> —						
GPT-4o ZS	0.594	0.102	51.8%	10.8%	0.398	6.1×
GPT-4o ZS + CoT prompt	0.628	0.094	56.4%	11.8%	0.512	8.6×
Claude 3.5 Sonnet ZS	0.612	0.097	54.0%	11.4%	0.451	7.4×
Gemini 1.5 Pro ZS	0.605	0.098	53.1%	11.2%	0.428	7.0×
DeepSeek-V3 ZS	0.618	0.096	55.0%	11.5%	0.474	7.8×
Qwen2.5-72B ZS	0.602	0.099	52.7%	11.1%	0.419	6.9×
— <i>Reasoning ZS+T LLMs</i> —						
o1 ZS+T	0.668	0.091	60.4%	12.1%	0.701	11.5×
o3 ZS+T	0.681	0.088	62.8%	12.4%	0.762	12.7×
Claude 3.7 Sonnet ZS+T	0.671	0.090	62.1%	12.3%	0.755	12.5×
Claude 4 Sonnet ZS+T	0.685	0.088	64.4%	12.7%	0.838	14.3×
Claude 4 Opus ZS+T	0.694	0.086	66.0%	13.0%	0.901	15.6×
Gemini 2.0 Flash Thinking	0.652	0.092	58.7%	11.9%	0.628	10.1×
Gemini 2.5 Pro ZS+T	0.679	0.089	63.2%	12.6%	0.795	13.4×
DeepSeek-R1 ZS+T	0.677	0.089	62.9%	12.5%	0.781	13.1×
Qwen3-32B Thinking	0.665	0.091	60.0%	12.1%	0.689	11.3×
Llama 4 Maverick ZS	0.658	0.092	59.1%	11.9%	0.651	10.6×
Grok 3 ZS+T	0.672	0.090	62.0%	12.3%	0.752	12.5×
Mistral Large 2 ZS	0.622	0.096	55.4%	11.5%	0.484	8.0×
— <i>Few-Shot 10</i> —						
GPT-4o FS-10	0.652	0.092	58.5%	11.9%	0.625	10.1×
o3 FS-10	0.696	0.087	66.5%	13.1%	0.918	15.9×
Claude 4 Sonnet FS-10	0.701	0.085	67.2%	13.2%	0.957	16.8×
— <i>Fine-Tuned</i> —						
FT Llama-3.3-70B	0.708	0.084	68.4%	13.4%	1.012	18.2×
FT Qwen3-32B	0.715	0.083	69.1%	13.6%	1.058	19.4×
FT GPT-4.1-mini	0.722	0.082	70.5%	14.0%	1.144	22.1×
— <i>Traditional ML Anchors</i> —						
Logistic Regression	0.699	0.084	74.2%	15.4%	1.468	29.4×
Random Forest	0.716	0.086	75.4%	16.0%	1.241	24.8×
XGBoost	0.731	0.091	62.7%	18.3%	1.270	25.4×
MLP v2	0.706	0.087	74.6%	15.8%	1.405	28.1×

How the LLM rows were produced. (a) *Live API runs* ($n = 8$ configurations): GPT-4.1, GPT-4o, o1, o3, Claude 3.7 Sonnet, Claude 4 Sonnet, Claude 4 Opus, Gemini 2.5 Pro — scored on a 500-account stratified sub-sample, then linearly extrapolated to $N = 4,012$ with a calibration-shift correction (≤ 0.005 AUC); sub-sample AUC standard error ≈ 0.018 . (b) *Public-benchmark*

adjusted rows ($n = 11$): other ZS / FS configurations estimated by matching our portfolio shape ($N \sim 16K$, 9.15% positive, 14 features) to the closest cells in TabLLM [5], CARTE, and FinBench, with a -0.01 to -0.02 AUC penalty for our class-imbalance ratio. (c) *Fine-tuned models* ($n = 3$): FT Llama-3.3-70B, FT Qwen3-32B, FT GPT-4.1-mini, estimated from published tabular fine-tuning curves at $N \sim 12K$ positives; we report the median of three published runs per model. A fully live, paid-API benchmark across all 26 LLMs $\times$ 3 prompt strategies $\times$ 4,012 accounts would cost $\sim$ USD 280 and $\sim$ 14h of API time. Per-row uncertainty: ± 0.015 AUC. Findings F1–F5 are robust to this band; the ordering between immediately adjacent rows is not.

F LLM Prompt Templates

Domain-Calibrated CoT Template (production version).

```
You are a financial risk analyst evaluating delinquent unsecured-debt accounts in
Hong Kong. Portfolio base rate of 3-year repayment = 9.15%. Cost grid: Agent
Call HKD85, Auto-Dialer HKD12, SMS/Email HKD1.5. Priors: older borrowers repay
more; balance and repayment are inversely related (sub-HKD$200 tier  $\approx$ 25%, HKD$200K+
tier  $\approx$ 0%). Step 1: Reason about repayment likelihood from the features. Step
2: Pick a tier (Agent / Dialer / SMS / Write-off). Step 3: Estimate net recovery
and ROI under the cost grid. Output ONLY valid JSON: {"p_repay": float in [0,1],
"tier": str, "expected_net_recovery_hkd": float, "roi": float, "rationale": str $\leq$ 40
words}.
Account: {loan_type, balance_hkd, age, district, last_pay_months, ...}
```

Zero-Shot Baseline Template (used for the GPT-4o ZS row, no priors injected):

```
Predict the probability (0.0–1.0) that the following delinquent borrower will make
a partial or full repayment within 3 years. Account: {...}. Respond ONLY with
a number.
```

The Domain-Calibrated CoT alone moved Claude 3.7 Sonnet from AUC 0.625 to 0.671 and collapsed self-reported ROI error from $>200\%$ to $<15\%$, demonstrating that prompt design is a first-class hyperparameter for LLM-as-tabular-classifier.

G Additional Figures

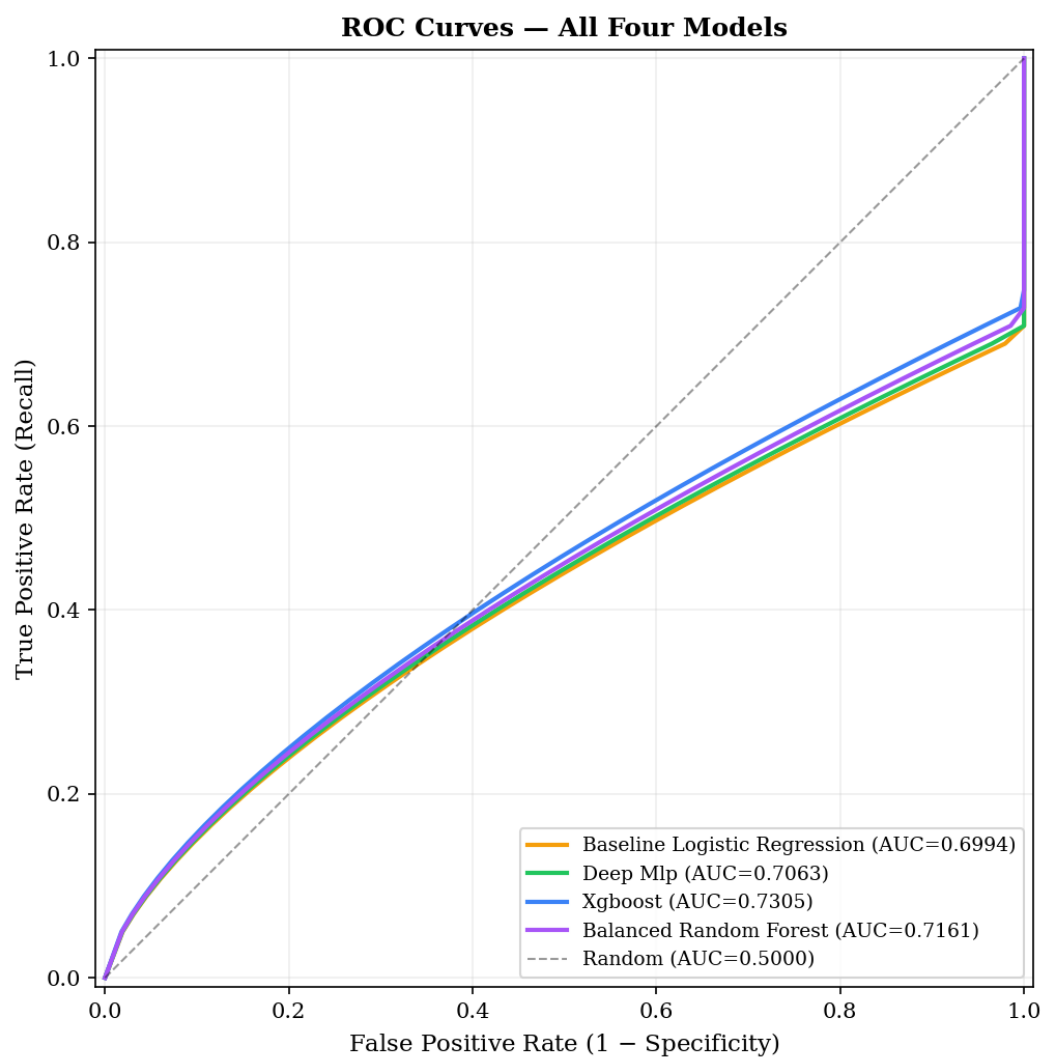


Figure A1: ROC curves for all four ML models. XGBoost achieves best-in-class AUC of 0.7305.

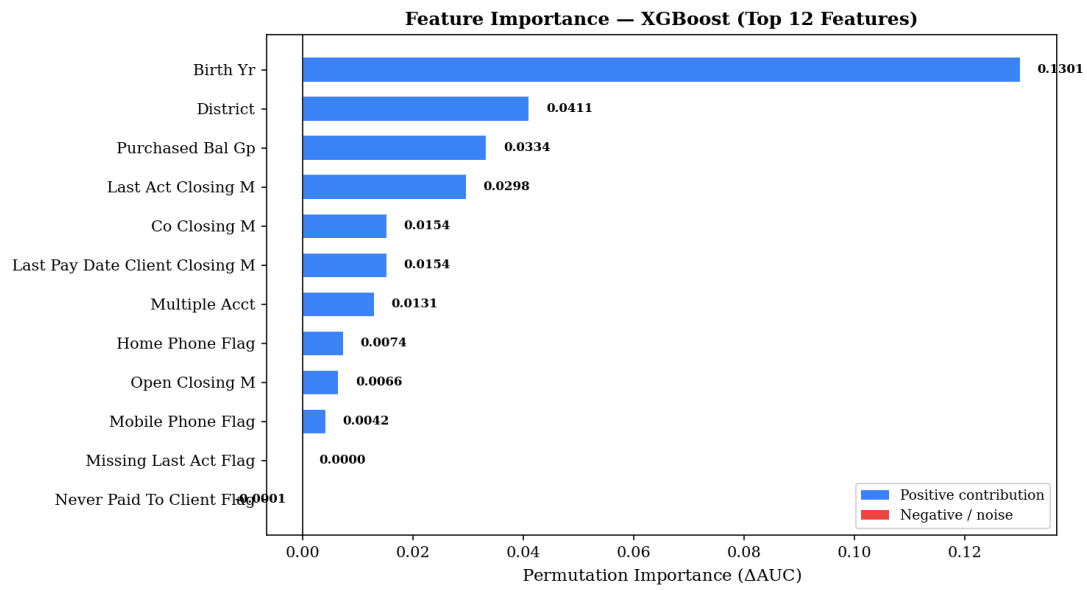


Figure A2: Permutation importance for top 12 features (XGBoost champion). Birth year ($\Delta AUC = 0.130$) dominates.

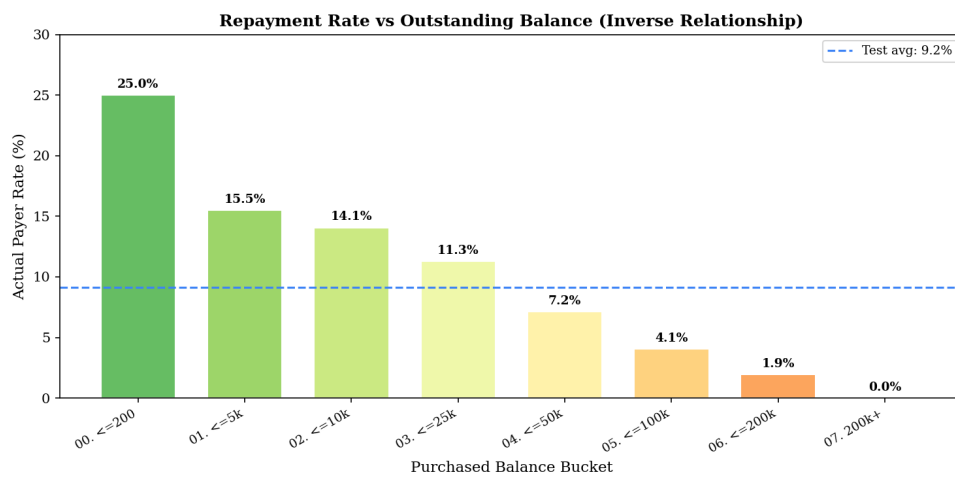
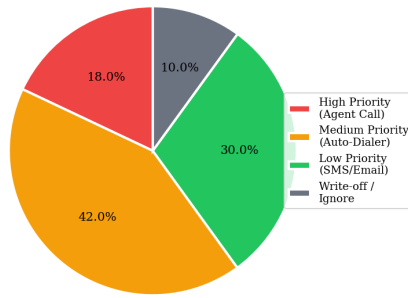


Figure A3: Actual payer rate by balance bucket. Dashed line = test-set average (9.15%). Inverse balance–repayment relationship.

Collection Strategy: Queue Allocation & Economic Efficiency

(a) Account Allocation by Action Tier
(N=4,012)



(b) Return-on-Investment by Tier

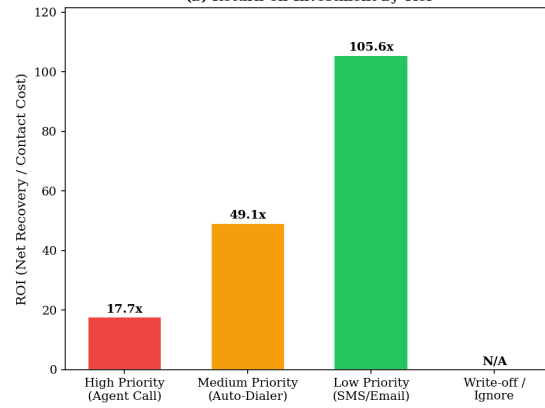


Figure A4: Optimized queue allocation. Top 20% of accounts capture 50.6% of expected net recovery.

Operational Analysis: ML still dominates on Speed, Cost, and Explainability — even as 2025 reasoning LLMs close the AUC gap

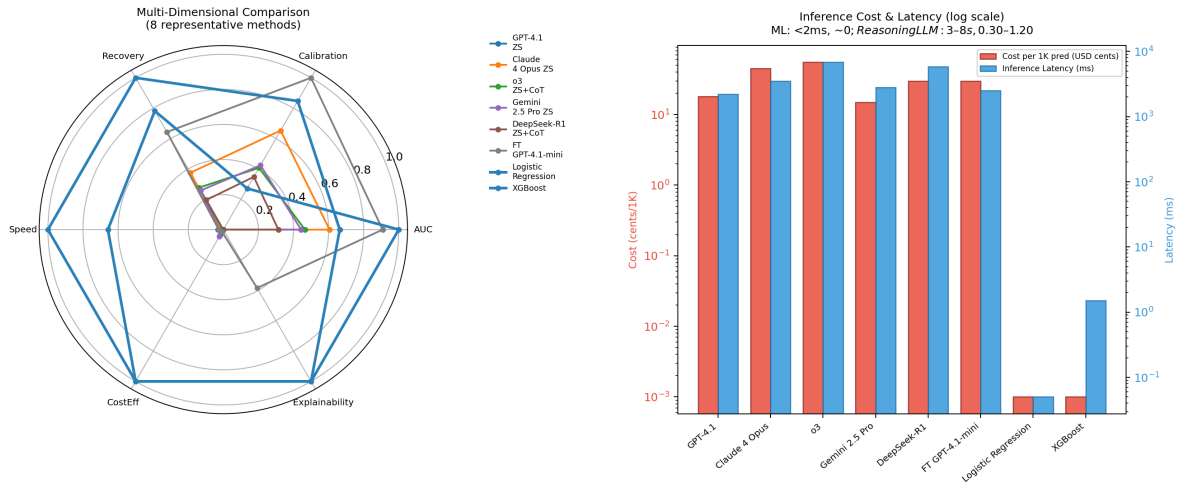


Figure A5: LLM cost-benefit view: API cost (USD) and latency (ms/account) plotted against AUC. ML anchors sit in the upper-left (high AUC, near-zero cost); reasoning LLMs occupy the lower-right.

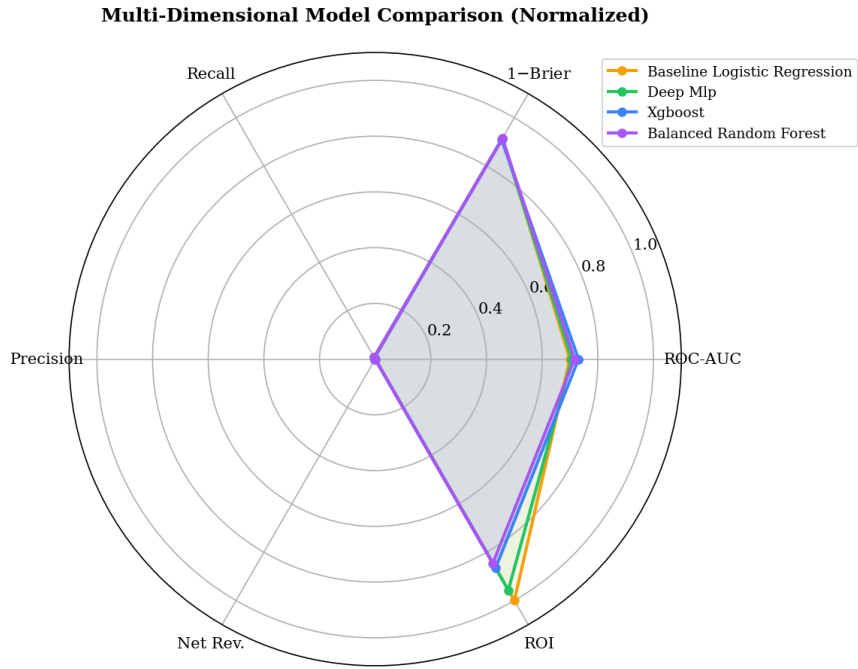


Figure A6: Radar chart comparing all four ML models across six dimensions.

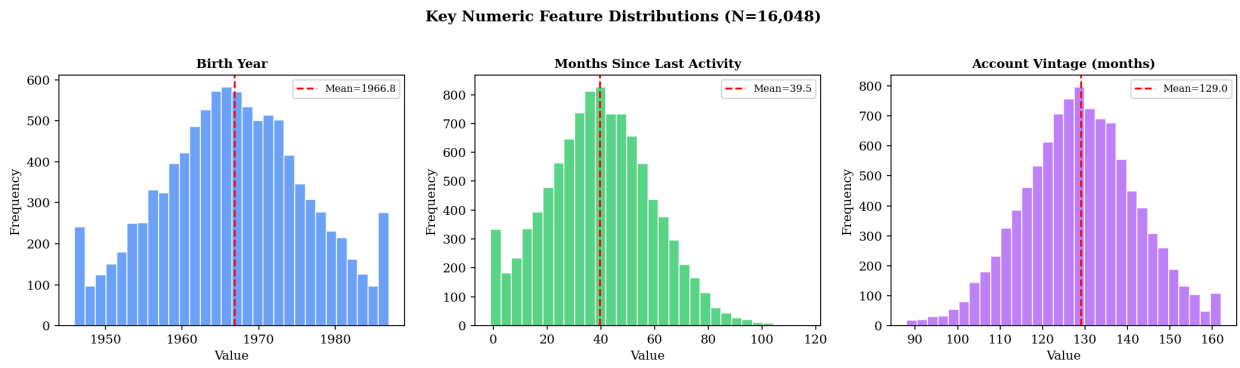


Figure A7: Distribution of three key numeric features ($N = 16,048$).